UNIVERSIDAD SANTO TOMAS
PRIMER CLAUSTRO UNIVERSITARIO DE COLOMBIA

Consultando la Base de Datos de una Universidad

Adrian Grateron, Harold Rodriguez y Jhonatan Smith Doncel

Facultad de estadística

Universidad Santo Tomas

Seccional Bogotá

Bases de Datos I

Carlos Isaac Zainea Maya

Mayo 2025

Contenido

Contenido 2
1. Introducción 3
2. Descripción general de las bases de datos 3
3. Desarrollo de consultas básicas 3
Pregunta 1. Lista el nombre y apellido de todos los estudiantes. 3
Pregunta 2. Muestra los detalles de los cursos que otorgan más de 3 créditos. 4
Pregunta 3. Obtén una lista de los profesores ordenados alfabéticamente por apellido. ... 4
4. Desarrollo de consultas de agregación 5
Pregunta 4. ¿Cuántos estudiantes hay en total? 5
Pregunta 5. ¿Cuál es el promedio de créditos de los cursos ofrecidos? 5
Pregunta 6. Muestra el número de estudiantes inscritos por cada curso. (GROUP BY). 6
5. Desarrollo de consultas multitabla y avanzadas 6
Pregunta 7. Muestra el nombre del estudiante y el nombre del curso para todas las inscripciones existentes. (INNER JOIN). 6
Pregunta 8. Lista todos los profesores y los nombres de los cursos que imparten. Incluye a los profesores que no imparten ningún curso. (LEFT JOIN). 7
Pregunta 9. Encuentra los estudiantes que están inscritos en el curso “Bases de Datos”. . 8
Pregunta 10. ¿Qué cursos tienen más de 5 estudiantes inscritos? (GROUP BY y HAVING). 9

Introducción

El presente informe desarrolla un conjunto de consultas SQL aplicadas a una base de datos relacional de una universidad. El objetivo es responder preguntas básicas, de agregación y multitabla mediante instrucciones DQL en PostgreSQL. Las consultas permiten explorar información sobre estudiantes, profesores, cursos e inscripciones, y refuerzan el uso de operaciones como SELECT, WHERE, ORDER BY, GROUP BY, INNER JOIN, LEFT JOIN y HAVING.

Para mejorar la legibilidad del entregable, los resultados se presentan como tablas formateadas dentro del documento. Esta versión también incorpora una corrección técnica sobre la consulta de estudiantes inscritos por curso, dado que la versión inicial agrupaba estudiantes por programa académico y no por curso.

Descripción general de las bases de datos

La base de datos contiene cinco tablas principales relacionadas mediante claves primarias y foráneas. La tabla inscripciones opera como tabla puente entre estudiantes y cursos, permitiendo modelar una relación muchos a muchos.

Tabla 1. Resumen del esquema de datos

Tabla	Registro	Descripción funcional
departamentos	6	Unidad académica o administrativa; se relaciona con profesores y cursos.
profesores	22	Docentes vinculados a departamentos; algunos no tienen curso asignado.
estudiantes	81	Población estudiantil registrada en la base.
cursos	16	Oferta de cursos con créditos, cupo máximo, profesor y departamento.
inscripciones	235	Relación estudiante-curso; contiene fecha, estado y nota final.

Relaciones principales: departamentos 1: N profesores; profesores 1: N cursos; departamentos 1: N cursos; estudiantes N:M cursos mediante la tabla inscripciones.

Desarrollo de consultas básicas

Pregunta 1. Lista el nombre y apellido de todos los estudiantes.

Consulta SQL:

```
SELECT
    nombres,
    apellidos
FROM estudiantes;
```

Resultado obtenido:

	nombres character varying (60)	apellidos character varying (60)
1	Sofia	Garcia
2	Mateo	Lopez
3	Valentina	Martinez
4	Santiago	Gonzalez
5	Isabella	Rodriguez
6	Sebastian	Perez
7	Camila	Sanchez
8	Daniel	Ramirez
9	Maria	Torres
10	Juan	Florez

Nota: se muestran las primeras 10 filas de 81 registros.

Interpretación. La consulta retorna los nombres y apellidos de los estudiantes registrados en la tabla estudiantes. El resultado permite identificar la población estudiantil cargada en la base de datos. Como no se especifica una cláusula ORDER BY, el orden de salida corresponde al orden en que PostgreSQL devuelve los registros.

Pregunta 2. Muestra los detalles de los cursos que otorgan más de 3 créditos.

Consulta SQL:

```
SELECT
     *
 FROM cursos
 WHERE creditos > 3;
```

Resultado obtenido:

	id_curso [PK] integer	codigo character varying (10)	nombre character varying (100)	creditos integer	semestre_oferta character varying (10)	cupo_maximo integer	id_profesor integer	id_departamento integer
1	2	CC201	Estructuras de Datos	4	2026-1	30	2	1
2	3	CC301	Bases de Datos	4	2026-1	30	1	1
3	5	CC501	Aprendizaje Automatico	4	2026-1	25	4	1
4	6	MA101	Calculo Diferencial	4	2026-1	40	5	2
5	8	MA301	Calculo Multivariado	4	2026-1	30	7	2
6	10	ES201	Inferencia Estadistica	4	2026-1	30	9	3
7	12	II101	Investigacion de Operacion...	4	2026-1	30	11	4

Interpretación. La consulta filtra los cursos cuyo número de créditos es superior a 3. En la base analizada, el resultado corresponde a 7 cursos de mayor carga académica relativa.

Pregunta 3. Obtén una lista de los profesores ordenados alfabéticamente por apellido.

Consulta SQL:

```
SELECT
     *
 FROM profesores
 ORDER BY apellidos, nombres;
```

Resultado obtenido:

	id_profesor [PK] integer	nombres character varying (60)	apellidos character varying (60)	correo character varying (120)	titulo_maximo character varying (20)	categoria character varying (20)	fecha_vinculacion date	id_departamento integer
1	17	Lucia	Acosta	lacosta@usta.edu.co	Maestria	Asistente	2022-01-20	6
2	18	Hernan	Buitrago	hbuitrago@usta.edu.co	Doctorado	Asociado	2011-08-01	1
3	10	Adriana	Castaneda	acastaneda@usta.edu....	Doctorado	Asistente	2021-07-01	3
4	7	Roberto	Diaz	rdiaz@usta.edu.co	Doctorado	Asistente	2019-02-15	2
5	19	Mariana	Espinosa	mespinosa@usta.edu.co	Maestria	Auxiliar	2024-01-15	3
6	20	Ivan	Forero	iforero@usta.edu.co	Especializacion	Auxiliar	2025-08-01	4
7	4	Laura	Gomez	lgomez@usta.edu.co	Doctorado	Asociado	2016-07-01	1
8	21	Natalia	Gutierrez	ngutierrez@usta.edu.co	Doctorado	Asociado	2016-02-01	6
9	6	Patricia	Hernandez	phernandez@usta.edu....	Maestria	Asociado	2012-09-01	2
10	8	Sandra	Lopez	slopez@usta.edu.co	Doctorado	Titular	2008-08-01	3
11	5	Jorge	Martinez	jmartinez@usta.edu.co	Doctorado	Titular	2005-03-10	2
12	14	Camilo	Mendoza	cmendoza@usta.edu.co	Maestria	Asistente	2019-08-01	5
13	22	Oscar	Pardo	opardo@usta.edu.co	Posdoctorado	Titular	2003-08-15	2
14	3	Andres	Pena	apena@usta.edu.co	Maestria	Asistente	2020-01-20	1
15	16	Fernando	Quintero	fquintero@usta.edu.co	Doctorado	Titular	2009-07-01	6
16	9	Felipe	Ramirez	framirez@usta.edu.co	Maestria	Asociado	2015-01-15	3
17	15	Beatriz	Restrepo	brestrepo@usta.edu.co	Doctorado	Asociado	2013-02-15	5
18	2	Maria	Rodriguez	mrodriguez@usta.edu.co	Doctorado	Titular	2010-08-15	1
19	13	Ricardo	Suarez	rsuarez@usta.edu.co	Doctorado	Titular	2007-01-20	4
20	11	Miguel	Torres	mtorres@usta.edu.co	Doctorado	Asociado	2014-02-01	4
21	12	Diana	Vargas	dvargas@usta.edu.co	Maestria	Asistente	2017-08-15	4
22	1	Carlos	Zainea	cizainea@usta.edu.co	Doctorado	Asociado	2018-02-01	1

Interpretación. La consulta lista los profesores registrados y los organiza alfabéticamente por apellido mediante ORDER BY. Este orden facilita la búsqueda y revisión de docentes, independientemente del identificador interno asignado en la base de datos.

Desarrollo de consultas de agregación

Pregunta 4. ¿Cuántos estudiantes hay en total?

Consulta SQL:

```
SELECT
     COUNT (*) AS total_estudiantes
 FROM estudiantes;
```

Resultado obtenido:

	total_estudiantes bigint
1	81

Interpretación. La consulta cuenta el número total de registros existentes en la tabla estudiantes. El resultado indica que la base contiene 81 estudiantes, sin distinguir si están o no inscritos en algún curso.

Pregunta 5. ¿Cuál es el promedio de créditos de los cursos ofrecidos?

Consulta SQL:

```
SELECT
     ROUND (AVG (creditos)::numeric, 2) AS promedio_creditos
 FROM cursos;
```

Resultado obtenido

	promedio_creditos numeric
1	3.38

Interpretación. La consulta calcula el promedio de créditos de los cursos ofrecidos mediante la función AVG. El resultado aproximado de 3,38 créditos indica que la oferta académica se concentra principalmente entre cursos de 3 y 4 créditos.

Pregunta 6. Muestra el número de estudiantes inscritos por cada curso. (GROUP BY).

Consulta SQL:

```
SELECT
     c.nombre AS curso,
     COUNT(i.id_estudiante) AS estudiantes_inscritos
 FROM cursos c
 LEFT JOIN inscripciones i
     ON c.id_curso = i.id_curso
 GROUP BY c.id_curso, c.nombre
 ORDER BY estudiantes_inscritos DESC;
```

Resultado obtenido:

	curso character varying (100)	estudiantes_inscritos bigint
1	Estadistica Descriptiva	36
2	Calculo Diferencial	35
3	Etica Profesional	30
4	Introduccion a la Programaci...	20
5	Inferencia Estadistica	20
6	Bases de Datos	17
7	Investigacion de Operaciones	11
8	Algebra Lineal	11
9	Estructuras de Datos	10
10	Modelos de Regresion	9
11	Gestion de la Calidad	9
12	Simulacion de Sistemas	7
13	Fundamentos de Administra...	7
14	Aprendizaje Automatico	7
15	Calculo Multivariado	4
16	Mineria de Datos	2

Interpretación. La consulta agrupa las inscripciones por curso y cuenta cuántos estudiantes están asociados a cada uno. El resultado permite identificar los cursos con mayor y menor demanda.

Desarrollo de consultas multitabla y avanzadas

Pregunta 7. Muestra el nombre del estudiante y el nombre del curso para todas las inscripciones existentes. (INNER JOIN).

Consulta SQL:

```
SELECT
     CONCAT(e.nombres, ' ', e.apellidos) AS estudiante,
```

```
    c.nombre AS curso
FROM inscripciones i
INNER JOIN estudiantes e
    ON i.id_estudiante = e.id_estudiante
INNER JOIN cursos c
    ON i.id_curso = c.id_curso
ORDER BY estudiante, curso;
```

Resultado obtenido:

	estudiante text	curso character varying (100)
1	Adrian Ibarra	Algebra Lineal
2	Adrian Ibarra	Calculo Diferencial
3	Alejandro Moreno	Algebra Lineal
4	Alejandro Moreno	Calculo Diferencial
5	Alejandro Moreno	Calculo Multivariado
6	Ana Cifuentes	Calculo Diferencial
7	Ana Cifuentes	Estadistica Descriptiva
8	Ana Cifuentes	Introduccion a la Programaci...
9	Andrea Gomez	Calculo Diferencial
10	Andrea Gomez	Estadistica Descriptiva

Nota: se muestran las primeras 10 filas de 235 registros

Interpretación. La consulta combina las tablas inscripciones, estudiantes y cursos mediante INNER JOIN. El resultado muestra cada relación estudiante-curso registrada en la base. Como se utiliza INNER JOIN, solo aparecen las inscripciones que tienen correspondencia válida tanto con un estudiante como con un curso.

Pregunta 8. Lista todos los profesores y los nombres de los cursos que imparten. Incluye a los profesores que no imparten ningún curso. (LEFT JOIN).

Consulta SQL:

```
SELECT
    CONCAT (p.nombres, ' ', p.apellidos) AS profesor,
    COALESCE (c.nombre, 'Sin curso asignado') AS curso
FROM profesores p
LEFT JOIN cursos c
    ON p.id_profesor = c.id_profesor
ORDER BY profesor, curso;
```

Resultado obtenido:

	profesor text	curso character varying
1	Adriana Castane...	Modelos de Regresion
2	Andres Pena	Mineria de Datos
3	Beatriz Restrepo	Sin curso asignado
4	Camilo Mendoza	Fundamentos de Administra...
5	Carlos Zainea	Bases de Datos
6	Carlos Zainea	Introduccion a la Programaci...
7	Diana Vargas	Gestion de la Calidad
8	Felipe Ramirez	Inferencia Estadistica
9	Fernando Quintero	Etica Profesional
10	Hernan Buitrago	Sin curso asignado
11	Ivan Forero	Sin curso asignado
12	Jorge Martinez	Calculo Diferencial
13	Laura Gomez	Aprendizaje Automatico
14	Lucia Acosta	Sin curso asignado
15	Maria Rodriguez	Estructuras de Datos
16	Mariana Espinosa	Sin curso asignado
17	Miguel Torres	Investigacion de Operaciones
18	Natalia Gutierrez	Sin curso asignado
19	Oscar Pardo	Sin curso asignado
20	Patricia Hernand...	Algebra Lineal
21	Ricardo Suarez	Simulacion de Sistemas
22	Roberto Diaz	Calculo Multivariado
23	Sandra Lopez	Estadistica Descriptiva

Interpretación. La consulta utiliza LEFT JOIN para listar todos los profesores y los cursos que imparten. Esta operación conserva todos los registros de la tabla profesores, incluso cuando no existe un curso asociado. Por esta razón, algunos profesores aparecen con la etiqueta Sin curso asignado.

Pregunta 9. Encuentra los estudiantes que están inscritos en el curso “Bases de Datos”.

Consulta SQL:

```
SELECT
    CONCAT(e.nombres, ' ', e.apellidos) AS estudiante,
    c.nombre AS curso
 FROM inscripciones i
 INNER JOIN estudiantes e
    ON i.id_estudiante = e.id_estudiante
 INNER JOIN cursos c
    ON i.id_curso = c.id_curso
 WHERE c.nombre = 'Bases de Datos'
 ORDER BY estudiante;
```

Resultado obtenido:

	estudiante text	curso character varying (100)
1	Andres Cortes	Bases de Datos
2	Camilo Bohorqu...	Bases de Datos
3	Carolina Echeve...	Bases de Datos
4	Daniel Ramirez	Bases de Datos
5	Felipe Rojas	Bases de Datos
6	Lucia Vega	Bases de Datos
7	Maria Torres	Bases de Datos
8	Mateo Lopez	Bases de Datos
9	Mauricio Urrego	Bases de Datos
10	Oscar Marin	Bases de Datos
11	Pablo Cardenas	Bases de Datos
12	Paula Herrera	Bases de Datos
13	Quintin Yanez	Bases de Datos
14	Sebastian Perez	Bases de Datos
15	Sofia Garcia	Bases de Datos
16	Tomas Mendez	Bases de Datos
17	Xavier Franco	Bases de Datos

Interpretación. La consulta identifica los estudiantes inscritos en el curso Bases de Datos. Para ello se relacionan las tablas estudiantes, inscripciones y cursos, y posteriormente se filtra el resultado con la condición WHERE c.nombre = 'Bases de Datos'.

Pregunta 10. ¿Qué cursos tienen más de 5 estudiantes inscritos? (GROUP BY y HAVING).

Consulta SQL:

```
SELECT
     c.nombre AS curso,
     COUNT(i.id_estudiante) AS estudiantes_inscritos
 FROM cursos c
 INNER JOIN inscripciones i
     ON c.id_curso = i.id_curso
 GROUP BY c.id_curso, c.nombre
 HAVING COUNT(i.id_estudiante) > 5
 ORDER BY estudiantes_inscritos DESC;
```

Resultado obtenido:

	curso character varying (100)	estudiantes_inscritos bigint
1	Estadistica Descriptiva	36
2	Calculo Diferencial	35
3	Etica Profesional	30
4	Introduccion a la Programaci...	20
5	Inferencia Estadistica	20
6	Bases de Datos	17
7	Investigacion de Operaciones	11
8	Algebra Lineal	11
9	Estructuras de Datos	10
10	Modelos de Regresion	9
11	Gestion de la Calidad	9
12	Simulacion de Sistemas	7
13	Aprendizaje Automatico	7
14	Fundamentos de Administra...	7

Interpretación. La consulta agrupa las inscripciones por curso, cuenta el número de estudiantes inscritos y utiliza HAVING para conservar únicamente los cursos con más de cinco estudiantes. Este resultado permite identificar los cursos con mayor nivel de participación dentro de la base de datos.

Conclusiones

El desarrollo de este trabajo nos permitió entender de una manera mucho más práctica cómo funcionan las bases de datos relacionales y la importancia que tienen al momento de organizar y consultar información. Aunque muchas veces SQL puede verse solo como un lenguaje para hacer consultas, durante el ejercicio pudimos notar cómo realmente ayuda a relacionar datos y obtener información útil de manera mucho más ordenada y eficiente.

A medida que se fueron realizando las consultas, también se hizo más claro el funcionamiento de las relaciones entre tablas mediante claves primarias y foráneas. Consultas como los INNER JOIN y LEFT JOIN ayudaron a comprender cómo se conectan estudiantes, cursos, profesores e inscripciones dentro de una misma base de datos, permitiendo obtener resultados más completos y fáciles de analizar.

También, funciones como COUNT, AVG, GROUP BY y HAVING mostraron cómo SQL no solo sirve para visualizar datos, sino también para generar análisis que pueden apoyar la toma de decisiones. Por ejemplo, fue posible identificar la cantidad de estudiantes registrados, conocer cuáles cursos tienen mayor demanda para proyecciones y presupuestos en próximos semestres y analizar la carga académica promedio de la oferta de cursos.

Consideramos que este tipo de ejercicios ayudan bastante a pasar de la teoría a algo más aplicado. Muchas veces los conceptos del modelo relacional parecen abstractos, pero al trabajar

directamente con consultas y resultados reales se entiende mucho mejor la importancia de tener una base de datos bien estructurada. Finalmente, este trabajo nos permitió fortalecer habilidades en PostgreSQL y comprender que una buena organización de los datos facilita no solo el almacenamiento de información, sino también su análisis y aprovechamiento en contextos reales.